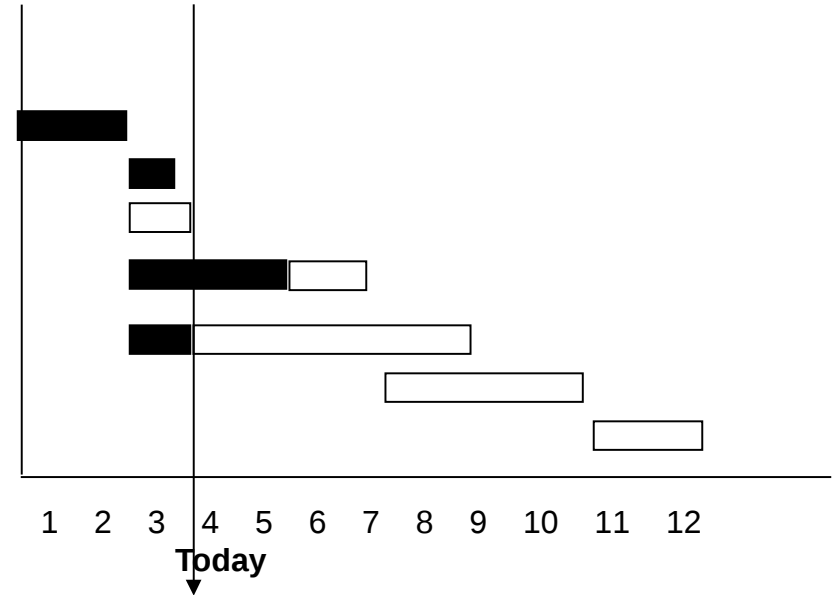


Project Management

Time-scale Charts

TASKS	
A	Study current email system
B	Define end-user requirements
C	Design Class diagram
D	Acquire computer technology
E	Plan & code email modules
F	Acceptance test new system
G	Deliver new system



Questions: From the above, can you guess:

- Which, if any, tasks should have been completed by today and aren't even started? _____
- Which, if any, tasks have been completed? _____
- Which, if any, tasks have been completed ahead of schedule? _____
- Which, if any, tasks are on or ahead of schedule? _____
- Which, if any, tasks are behind schedule? _____

Building and Using a Time-scale Chart

Steps for building a Time-scale Chart

1. Identify the tasks to be scheduled
2. Determine the durations of each task
3. List each task down the vertical axis of chart

In general, list tasks to be performed first at the top and then move downward as the tasks will happen

4. Use horizontal axis for the dates/time
5. Determine start and finish dates/time for activities

Consider which tasks must be completed or partially completed before the next task

To use the Time-scale chart to report progress:

- If the task has been completed, completely shade in the bar corresponding to the task
- If the task has been partially completed, shade in the percentage of the bar that represents the percentage of the task that has been completed
- Unshaded bars represents tasks that have not been started.

Game Development Exercise

Find the critical path using the PERT chart. Use any tool .

TASK	DURATION (days)	PREDECESSORS
A Graphics Engine	14	-
B Sound Engine	5	I
C Music Engine	5	J
D Input Engine	10	A
E Game play / general programming	31	B, C, D
F Physics	7	E
G 2D Artwork	14	-
H 3D Artwork	21	G
I Sound Effects	14	-
J Music	9	-
K Level Design	21	F, H

Time-scale Chart: Exercise

Task	Prec Tasks	Description	Time(hrs)
A	none	decide on date for party	1
B	A	book bouncy castle	1
C	A	send invitations	4
D	C	receive replies	7
E	D	buy toys and balloons	1
F	D	buy food	3
G	E	blow up balloons	2
H	F	make food	1
I	H, G	decorate	1
J	B	get bouncy castle	1
K	J, I	have party	1
L	K	clean up	4
M	K	send back bouncy castle	1
N	L	send thank you letters	3
O	M	donate unwanted gifts	3

PERT vs. Time-scale chart

PERT chart

- All us to **show dependencies** explicitly
- Allow us to **calculate critical path**
- Can tell us **how one task falling behind affects other tasks**

Time-scale charts

- Allow us to **record progress** of project
- Allow us to see **what tasks are falling behind**
- Allow us to **represent overlapping tasks**

Earned Value Management

- How much work you planned to have accomplished by now (in currency or hours) called the **Planned Value**
- How much you have actually spent by now (in currency or hours), called **Actual Cost**
- The value (in terms of your baseline budget) of the work accomplished by now (in currency or hours), called the **Earned Value!**

Idea is to link schedule and cost together to monitor both in the same “units” of value

Earned Value Management

- Planned value (PV) - Value of all resources needed to meet the project's objectives
 - Each objective of a project has an associated PV.
- Budgeted (cost) At completion (BAC) - Sum of all the PVs
- Earned value (EV) - Amount of value completed at any point during the project
- Actual Cost (AC) - actual amount of money you have spent so far.
 - In a **perfect project**, AC and EV are the same.

--	--

Earned Value Management Example

- We've budgeted Rs. 200 to buy, setup, network and test a new system
 - Our planned values (PVs) of each task are:
 - Buy – Rs. 50, Setup – Rs. 75, network – Rs. 50, test – Rs. 25
 - Our **BAC** is therefore Rs. 200
- We've now completed phase one (Buy), and thus our earned value (EV) is now Rs. 50.
- To do this we spent Rs. 60 (our actual cost (AC))

EV Management – Few key metrics

■ Schedule Performance Index (**SPI**)

■ $EV / PV \rightarrow 50/50 = 1$ (perfect).

■ Our group is on schedule

■ Cost Performance Index (**CPI**)

■ $EV / AC \rightarrow 50/60 = 0.83$

■ For every Rs. 100 spent, I get Rs. 83 worth of work.

■ Estimated (cost) At Completion (**EAC**)

■ $BAC / CPI = 200 / 0.83 = 240.96$

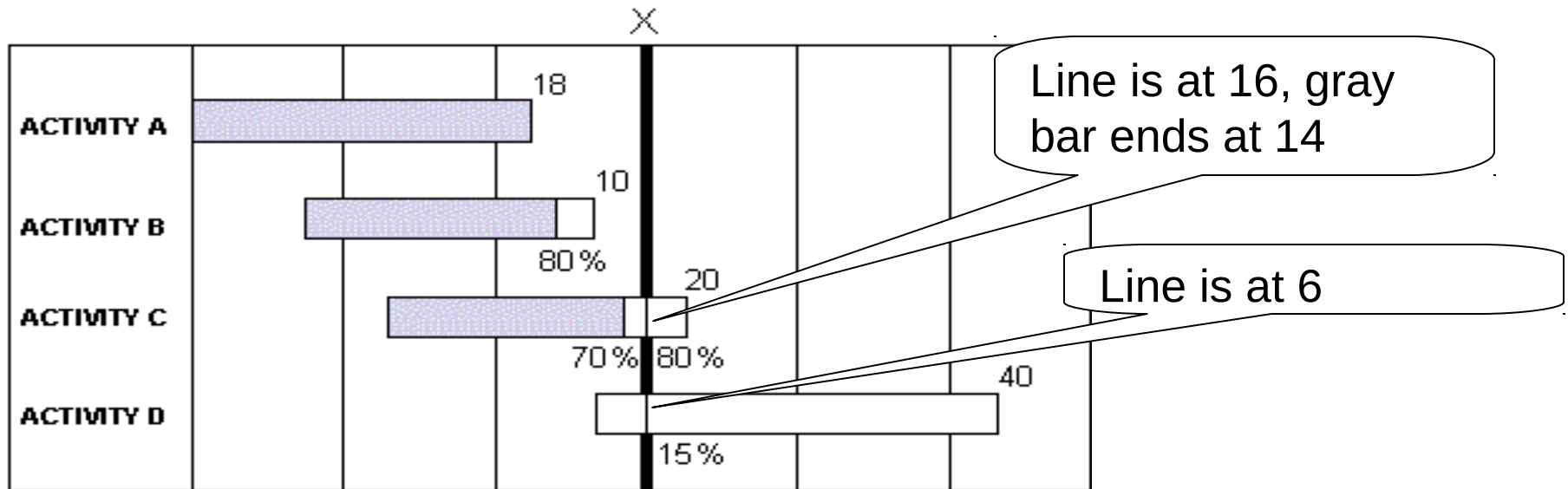
■ Schedule Variance (**SV**) : $EV - PV$

■ Cost Variance (**CV**) : $EV - AC$

EV Earned Value
PV Planned Value
AC Actual Cost

BAC Budgeted (cost) At completion

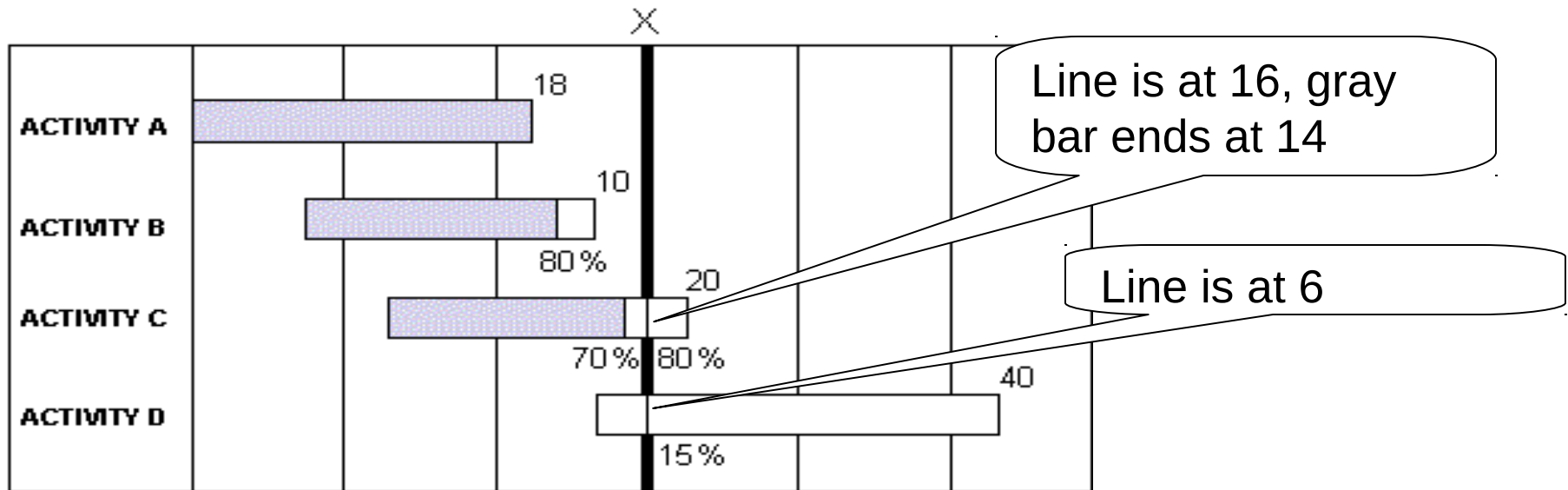
Example



Actual Cost (of work performed) = \$45 (Data from Acct. System)

- **Compute Planned Value (PV) at time X.**
- **Compute Earned Value (EV) at time X.**
- **Compute Schedule Variance and Schedule Performance Index.**

Solution to Example



- **PLANNED VALUE (Budgeted cost of the work scheduled)**
 $= 18 + 10 + 16 + 6 = \$50$

What is PV at time X?
- **EARNED VALUE (Budgeted cost of the work performed)**
 $= 18 + 8 + 14 + 0 = \$40$

What is EV at time X?
- **ACTUAL COST (of the work performed) = \$45 (Data from Acct. System)**
- Therefore:
 - **Schedule Variance** $= EV - PV = 40 - 50 = -\$10$
 - **Schedule Performance Index** $= EV / PV = 40 / 50 = 0.8$

Project Scope and Risk

Scope Creep

- The scope of your project is all the work you initially planned to do.
- Scope creep is,
 - when your project gets new tasks throughout its lifetime **without adding more resources** to handle new tasks.
 - The scope is “creeping” up...
- **Scope changes are OK**, and really unavoidable... that's fine.
 - However, you must **update the resources** (time, features or people accordingly)

--	--

Scope Change versus Creep

Your company has a \$1million dollar contract with a defined scope.

Scope change:

Customer: please add all these requirements, and I'll increase the contract to \$2 million dollars

Manager: Certainly! 😊

Scope creep:

Customer: please add all these requirements, and I'll be really happy.

Manager: Certainly! 😞

Managing Scope

- How to deal with the inevitable “Scope creep”?
 - Joint Application Development and prototyping
 - Formal change approval
 - Defer additional requirements as future system enhancements

Managing Risk

- Document your risks in a risk management plan
 - 1 Description of risk
 - 2 Likelihood of occurrence (0-100%)
 - 3 Impact - 1(low) → 5 (high), or cost (in currency)
 - 4 Exposure = Impact * Likelihood
 - 5 Mitigation strategy
 - How to **lessen the impact** of the risk?
 - How to **lessen the likelihood**?
 - An **action plan**, if risk occurs
- Update and track your risks.
- Communicate your risks to upper management

Software Projects get into trouble when...

- People in s/w development **don't understand their customer's needs.**
- The product **scope is poorly defined.**
- **Changes are managed poorly.**
- The chosen **technology changes.**
- **Business needs change** [or are **ill-defined**].
- **Deadlines are unrealistic.**
- Users are resistant.
- Sponsorship is lost [or was never properly obtained].
- The **project team lacks people with appropriate skills.**
- **Managers** [and practitioners] **avoid best practices** and lessons learned.

Classic Mistakes

- Overly optimistic schedule
- Failing to monitor the schedule
- Failing to update the schedule
- Adding people to a late project
- Failure to manage expectations of others
- Leaving out a task

The above mistakes should be best checked and preferably avoided.

Common Technology-Related Mistakes

- Overestimated savings from new tools and methods
- Switching tools in mid-project
- Lack of automated source-code control

Project Approaches to Remember

- *Start on the right foot.* This is accomplished by working hard (very hard) to **understand the problem** that is to be solved and then setting realistic objectives and expectations.
- *Maintain momentum.* The project manager must **provide incentives** to keep turnover of personnel to an absolute minimum, the team should **emphasize quality** in every task it performs, and senior management should do everything possible to stay out of the team's way.
- *Track progress.* For a software project, progress is tracked as work products (e.g., **models, source code, sets of test cases**) are produced and approved (using formal **technical reviews**) as part of a quality assurance activity.
- *Make smart decisions.* In essence, the decisions of the project manager and the software team should be to "**keep it simple.**"
- *Conduct a postmortem analysis.* Establish a consistent mechanism for **extracting lessons learned** for each project.